

Provincia de Buenos Aires

Dirección General de Cultura y Educación

Dirección Provincial de Educación Secundaria



ESCUELA DE EDUCACIÓN SECUNDARIA TÉCNICA
N° 1

DE VICENTE LÓPEZ, "EDUARDO ADER"

Cerrito 3966, Villa Adelina (1605), Tel. 4735-0174

tecnica1vl@gmail.com

Proyecto:

Malvinas: Voces del frente

Curso y división:

7°2°

Integrantes:

Bruno Fornasar, Alan Bissio y Tomás Onesti

Tema:

Manual de programador

Índice

Introducción.....	3
Marco tecnológico.....	4
Unity.....	4
Visual Studio Code.....	4
Git.....	4
Organización de directorios.....	5
Instalación.....	5
Configuración de entorno.....	6
Abrir el proyecto en el editor de Unity.....	6
Abrir el proyecto en el editor de código.....	7
Explicación de scripts.....	9

Introducción

En el presente documento se brinda la información necesaria para poder realizar un relevamiento adecuado en el proyecto *“Malvinas - La batalla en el frente”*. Se informará acerca de la infraestructura del proyecto así como de las tecnologías utilizadas para llevarlo a cabo.

Este proyecto es un videojuego 2D desarrollado con el motor gráfico *“Unity”*.

Marco tecnológico

Para poder trabajar adecuadamente en el proyecto necesitamos de las siguientes herramientas:

- Unity versión 2022.3.62f3
- Visual Studio Code de Microsoft
- Git

Unity

Este es el motor gráfico que se ha utilizado para este proyecto, se ha elegido esta versión ya que la más reciente versión 6 está sujeta a posibles errores por su poco tiempo en el mercado. Su instalación se realiza desde el entorno “*Unity Hub*” donde debería aparecer como la última versión en la lista. En caso de que esto no ocurra puede consultar el sitio oficial de Unity para encontrar una lista con todas las versiones anteriores del motor. Una vez selecciona la indicada se le debería abrir automáticamente el entorno de Unity Hub.

Visual Studio Code

Como se ha indicado anteriormente, la versión que recomendamos es la de Microsoft, ya que esta cuenta con la extensión para desarrollo con Unity, la cual facilitará en gran medida el trabajo. En caso de contar con otra versión del editor que no cuente con esta extensión en la tienda, recomendamos que consulte la documentación de su editor que por lo general incluye una forma de usar extensiones exclusivas de Microsoft.

Git

Para el control de versiones utilizamos Git. Si se desea puede usar el control de versiones de Unity el cual debería dar los mismos resultados.

Organización de directorios

En esta sección se mostrará la estructura de directorios y qué carpetas y archivos son relevantes para el desarrollo.

- `/:` En este directorio encontrarás los archivos de configuración de Git como el `".gitignore"`.
- `/Assets/:` Este debe ser el directorio más importante de todo el proyecto. Este es en el que se encuentran todos los assets como imágenes y scripts de nuestro juego. Es muy importante ordenar todo en carpetas ya que todos los elementos se encuentran en este directorio.
- `/Assets/Scenes/:` En esta carpeta se muestran las escenas del videojuego.
- `/Assets/interface/:` En este directorio se encuentran todos los assets asociados con la interfaz de usuario.
- `/Assets/animation/:` En esta carpeta se pueden ver todas las animaciones agrupadas por carpetas. Cada una corresponde a una distinta.
- `/Assets/Settings/:` Esta carpeta guarda la configuración por defecto que crea Unity.
- `/Assets/Sprites/:` Este directorio es donde guardamos todos los sprites que lleguemos a usar. Lo ideal es agrupar los archivos por carpetas para mantener legible la estructura.
- `/Assets/script/:` En este directorio colocamos todos los scripts agrupados en carpetas en base a su finalidad. Por ejemplo, aquellos diseñados para controlar al jugador, están todos juntos dentro de la carpeta *"PlayerScripts"*.

Instalación

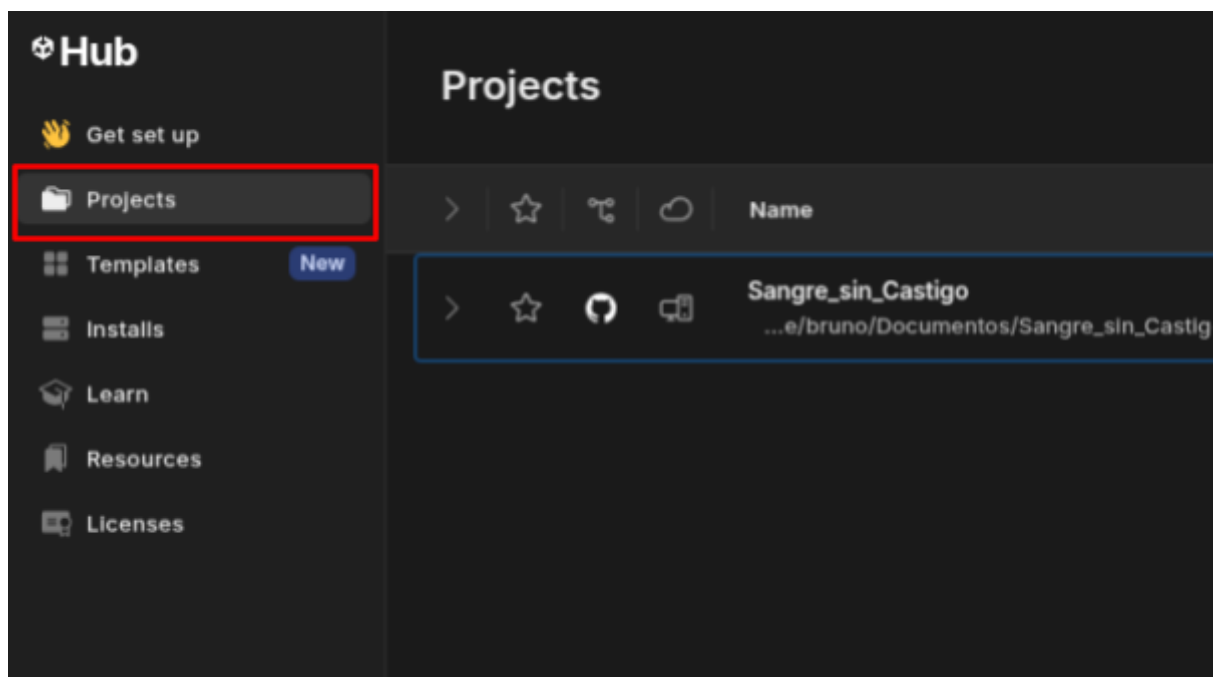
Para instalar el proyecto necesitamos tener una cuenta de Github colaboradora del proyecto para poder acceder al repositorio del mismo. Una vez tengamos acceso, podremos clonar el repositorio con git de la manera que se prefiera.

Configuración de entorno

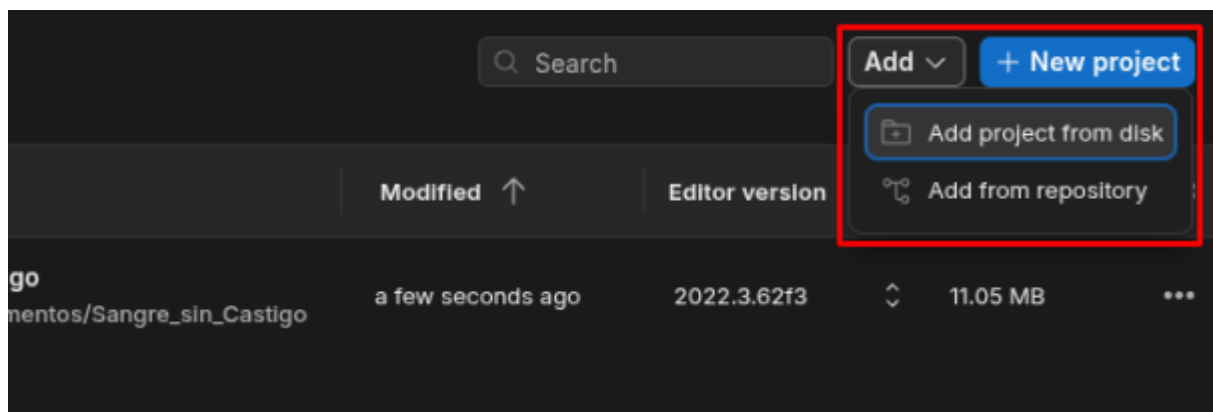
Una vez tenemos el proyecto, ya podemos preparar el entorno para comenzar a modificar el proyecto.

Abrir el proyecto en el editor de Unity

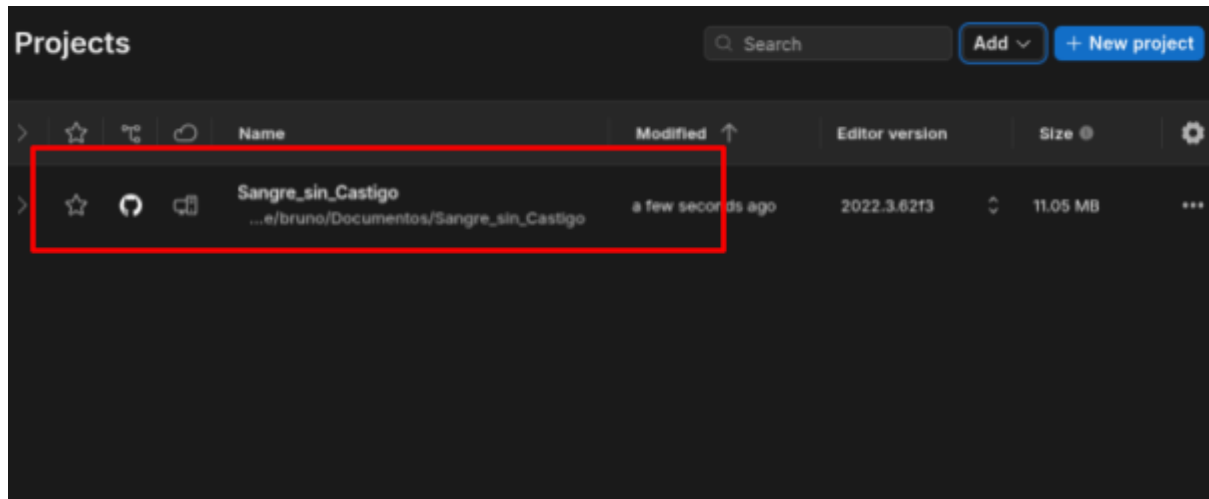
Para abrir el proyecto tenemos que abrir el panel de Unity Hub y dirigirnos a la sección de “Proyectos”.



En esta sección abriremos la pestaña “Añadir” y seleccionaremos “Añadir desde el disco”.



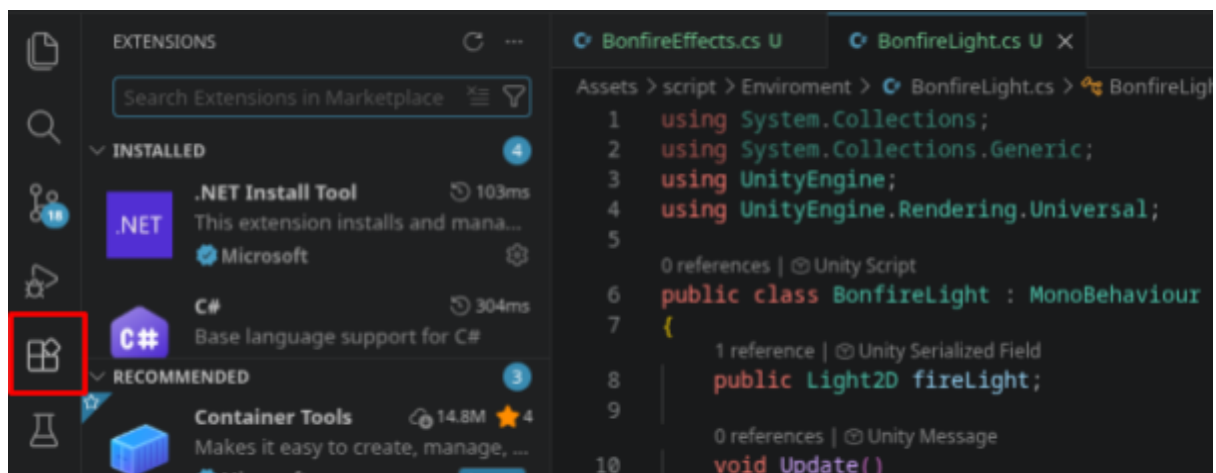
Una vez seleccionemos esta opción se nos abrirá nuestro explorador de archivos por defecto. En el mismo buscaremos la carpeta donde tengamos el proyecto y la seleccionaremos. Tras esto, el proyecto aparecerá en nuestro panel principal.



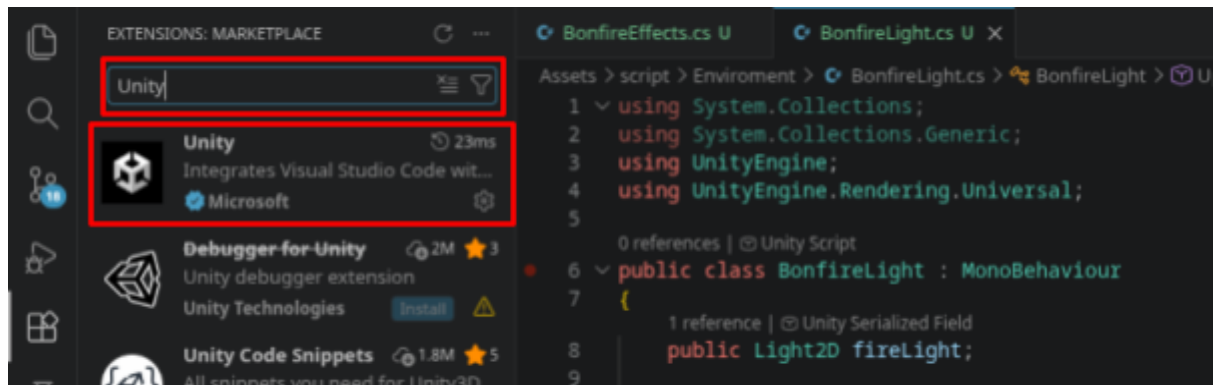
Tras esto ya podemos seleccionarlo para que se nos abra el editor con nuestro proyecto abierto.

Abrir el proyecto en el editor de código

Como Unity no cuenta con editor de código integrado, necesitamos uno externo. Ya especificamos los requisitos que utilizaremos Visual Studio Code. Para trabajar cómodamente con Unity necesitaremos la extensión oficial de Microsoft. Para lo cual deberemos abrir el editor y buscar la sección de extensiones.



En esta sección buscaremos la extensión de Unity y nos aseguraremos de que sea la de Microsoft.



Explicación de scripts

Script de movimiento “playermovent”. Script del movimiento del jugador, control de la velocidad del jugador y animaciones de movimiento del jugador.

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class topdownmovent : MonoBehaviour
6  {
7      //Solo con esto tenemos el movimiento del personaje
8      //Variables de referencia para la velocidad y direccion
9      [SerializeField] private float speed;
10     [SerializeField] private Vector2 direction;
11     private Rigidbody2D rb2D;
12     //Para la animacion
13     [SerializeField] private float movex;
14     [SerializeField] private float movey;
15     private Animator animator;
16
17     //Funcion para iniciar(siempre en mayuscula, son funciones de unity)
18     private void Start()
19     {
20         animator = GetComponent<Animator>();
21         rb2D = GetComponent<Rigidbody2D>();
22     }
23
24     //Funcion para actualizar(siempre en mayuscula)
25     private void Update()
26     {
27         movex = Input.GetAxisRaw("Horizontal");
28         movey = Input.GetAxisRaw("Vertical");
29         animator.SetFloat("movex", movex); //Animacion en x(izquierda y derecha)
30         animator.SetFloat("movey", movey); //Animacion en y(arriba y abajo)
31         //Parte del funcionamiento de la animacion idle(para estar quieto)
32         if (movex != 0 || movey != 0) {
33             animator.SetFloat("finx", movex);
34             animator.SetFloat("finy", movey);
35         }
36         direction = new Vector2(movex, movey).normalized;
37         // Debug.Log("movex: " + movex.ToString());
38         // Debug.Log("movey: " + movey.ToString());
39     }
40     //Otra funcion para actualizar(siempre en mayuscula)
41     private void FixedUpdate()
42     {
43         rb2D.MovePosition(rb2D.position + direction * speed * Time.fixedDeltaTime);
44     }
45 }
```

Script de disparo del jugador. Controla el cómo y con qué se dispara(velocidad, cantidad, delay de las balas).

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class controlCamara : MonoBehaviour
6  {
7      [SerializeField] private GameObject bulletP; //Prefab de la bala
8      [SerializeField] private float bulletspeed; //velocidad
9      [SerializeField] private float shootDelay; // un pequeño delay
10
11      private float lastShoot; //Detecta el ultimo disparo para poner el delay
12
13      void Start()
14      {
15      }
16
17      void Update()
18      {
19          float horShoot = Input.GetAxisRaw("ShootHorizontal"); //Teclas de disparo horizontal y abajo la vertical(alfinal lo hice con flechas, si quieren lo puedo cambiar)
20          float verShoot = Input.GetAxisRaw("ShootVertical");
21
22          if((horShoot != 0 || verShoot != 0) && Time.time > lastShoot + shootDelay) //Establece cuando se ejecuta el disparo teniendo en cuenta el delay
23          {
24              shoot(horShoot, verShoot); //Dispara
25              lastShoot = Time.time; //Guarda el ultimo disparo(para el delay de arriba(esto tambien por si hacemos varias armas lo vamos modificando y así))
26          }
27      }
28
29      void shoot(float x, float y)
30      {
31          GameObject bullet = Instantiate(bulletP, transform.position, transform.rotation) as GameObject; //instanciamos las balas
32          bullet.AddComponent<Rigidbody2D>().gravityScale = 0; //sin gravedad por ser un top-down
33          bullet.GetComponent<Rigidbody2D>().velocity = new Vector2(x, y) * bulletspeed; //la velocidad de las balas y direccion al disparar
34      }
35  }
```

Movimiento de los enemigos. Esto cambiará pronto(ya lo arreglo)

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class enemigo : MonoBehaviour
6 {
7     public Transform player; //Jugador
8     [SerializeField] private float detection; //Rango de detección
9     [SerializeField] private float enemySpeed; //Velocidad(hay que poner valores altos(supongo por el deltaTime))
10    private Rigidbody2D rb; //Enemigo
11    private Vector2 movement; //Movimiento
12
13    void Start()
14    {
15        rb = GetComponent<Rigidbody2D>();
16    }
17
18    void Update()
19    {
20        float distance = Vector2.Distance(transform.position, player.position); //Calcula la distancia
21        if(distance < detection)
22        {
23            Vector2 direction = (player.position - transform.position).normalized; //si la distancia es correcta le indica donde debe moverse ne base a las posiciones
24            movement = new Vector2(direction.x, direction.y); //Le dice que se mueva izquierda, derecha, arriba y abajo.
25        }else{
26            movement = Vector2.zero; //si sale del rango que no se mueva(aunque funciona ma o meno esto)
27        }
28
29        rb.MovePosition(rb.position + movement * enemySpeed * Time.deltaTime); //Lo mueve
30    }
31 }
```

Control de duración de las balas

Code Blame 27 lines (20 loc) · 428 Bytes

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class bulletController : MonoBehaviour
6 {
7     public float lifetime;
8
9     void Start()
10    {
11        StartCoroutine(DeathDelay());
12    }
13
14    // Update is called once per frame
15    void Update()
16    {
17    }
18
19
20    IEnumerator DeathDelay()
21    {
22        yield return new WaitForSeconds(lifetime);
23        Destroy(gameObject);
24    }
25
26 }
```